

Lecture-#15

Natural Language Processing

Language modeling

What Is A Language Model?

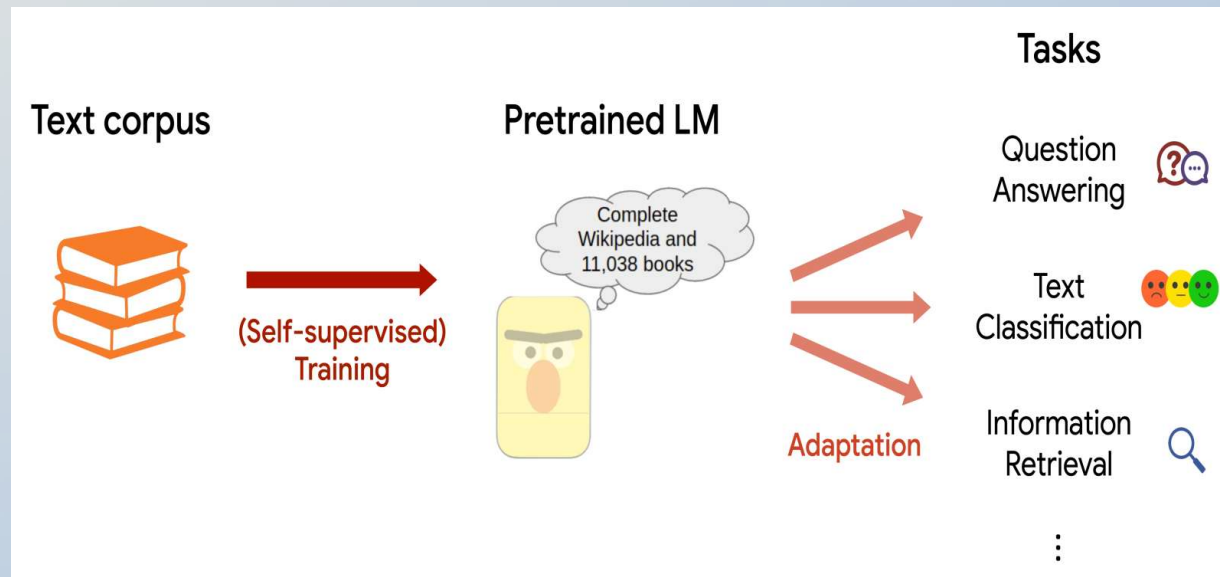
- A language model is a machine learning model designed to represent the language domain.
 - Language models analyze bodies of text data to provide a basis for their word predictions.
 - It can be used as a basis for a number of different language-based tasks, for instance:
 - ❖ [Question answering](#)
 - ❖ [Semantic search](#)
 - ❖ [Summarization](#)
- and plenty of other tasks that operate on natural language.

Probability distributions over sentences (i.e., word sequences)

$$P(W) = P(w_1 w_2 w_3 w_4 \dots w_k)$$

Language Models Play the Role of ...

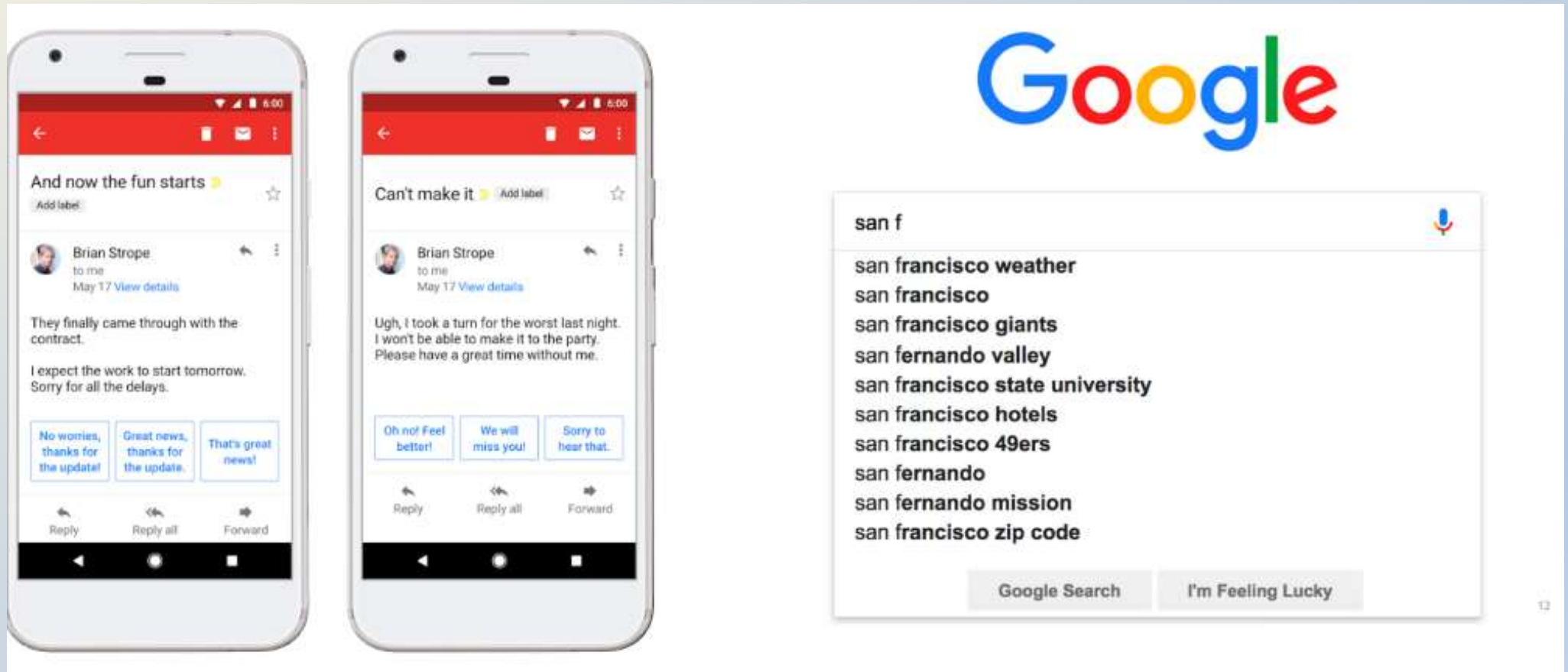
- ❖ A judge of grammaticality
- ❖ A judge of semantic plausibility
- ❖ An enforcer of stylistic consistency
- ❖ A repository of knowledge (?)



Applications of Language Model?

- Machine Translation
 - $p(\text{strong winds}) > p(\text{large winds})$
- Spell Correction
 - The office is about 15 minutes from my house
 - $p(15 \text{ minutes from my house}) > p(15 \text{ minuets from my house})$
- Speech Recognition
 - $p(\text{I saw a van}) \gg p(\text{eyes awe of an})$
- Summarization, question-answering, handwriting recognition, etc..

Applications of Language Model?



The image displays two smartphones on the left, each showing an email conversation. The left phone shows an email from Brian Strope with the subject 'And now the fun starts' and a response with three suggested replies: 'No worries, thanks for the update!', 'Great news, thanks for the update.', and 'That's great news!'. The right phone shows an email from Brian Strope with the subject 'Can't make it' and a response with three suggested replies: 'Oh not Feel better!', 'We will miss you!', and 'Sorry to hear that.'.

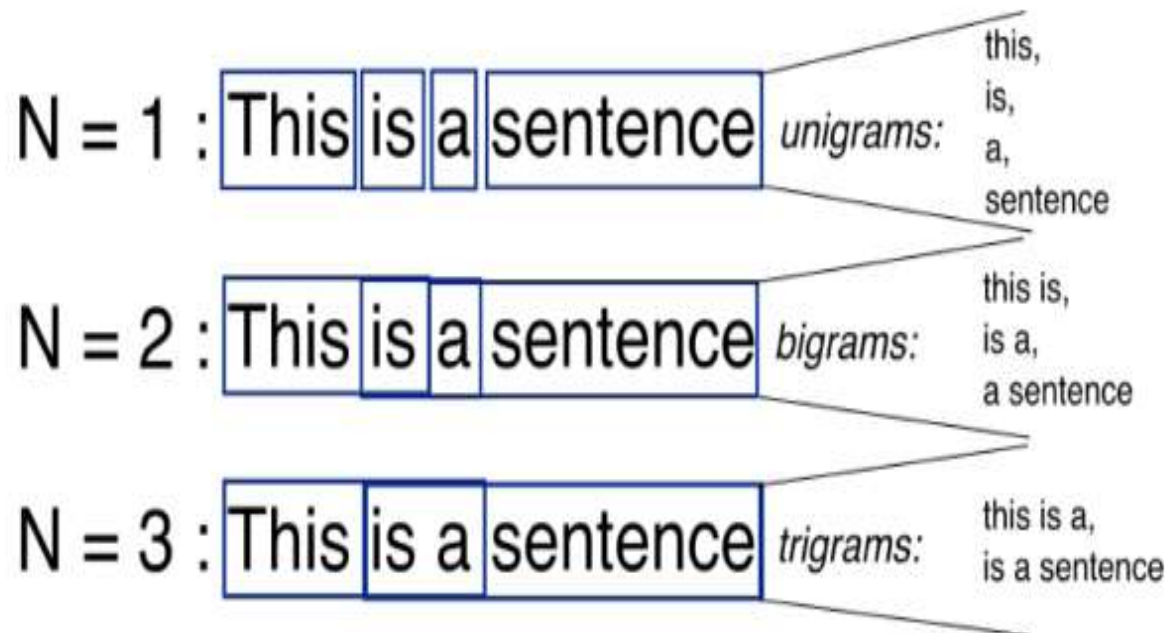
On the right, a Google search interface is shown with the search bar containing 'san f'. Below the search bar, a list of suggestions is displayed:

- san francisco weather
- san francisco
- san francisco giants
- san fernando valley
- san francisco state university
- san francisco hotels
- san francisco 49ers
- san fernando
- san fernando mission
- san francisco zip code

At the bottom of the search interface, there are two buttons: 'Google Search' and 'I'm Feeling Lucky'.

N-grams Model?

N-grams: a contiguous sequence of n tokens from a given piece of text



N-grams Model?

- Unigram model: $P(w_1)P(w_2)P(w_3) \dots P(w_n)$
- Bigram model: $P(w_1)P(w_2|w_1)P(w_3|w_2) \dots P(w_n|w_{n-1})$

- Trigram model:

$$P(w_1)P(w_2|w_1)P(w_3|w_2, w_1) \dots P(w_n|w_{n-1}w_{n-2})$$

- N-gram model:

$$P(w_1)P(w_2|w_1) \dots P(w_n|w_{n-1}w_{n-2} \dots w_{n-N})$$

N-grams Model?

Training Data:

Sentence: "The quick brown fox jumps over the lazy dog."

Tokenization: ["The", "quick", "brown", "fox", "jumps", "over", "the", "lazy", "dog"]

Word	Frequency	Probability
-----	-----	-----
The	2	0.222
quick	1	0.111
brown	1	0.111
fox	1	0.111
jumps	1	0.111
over	1	0.111
lazy	1	0.111
dog	1	0.111

N-grams Model?

Training Data:

Sentence: "The quick brown fox jumps over the lazy dog."

Tokenization: ["The", "quick", "brown", "fox", "jumps", "over", "the", "lazy", "dog"]

Word Frequency Calculation:

Word	Frequency
The	2
quick	1
brown	1
fox	1
jumps	1
over	1
lazy	1
dog	1

N-grams Model?

Training Data:

Sentence: "The quick brown fox jumps over the lazy dog."

Tokenization: ["The", "quick", "brown", "fox", "jumps", "over", "the", "lazy", "dog"]

Total Words (N):

N = 9

Unigram Probability Calculation:

$P(\text{word}) = \text{Frequency of word} / \text{Total number of words}$

Word	Frequency	Probability
-----	-----	-----
The	2	0.222
quick	1	0.111
brown	1	0.111
fox	1	0.111
jumps	1	0.111
over	1	0.111
lazy	1	0.111
dog	1	0.111

N-grams Model?

Training Data:

Sentence: "The quick brown fox jumps over the lazy dog."

Tokenization: ["The", "quick", "brown", "fox", "jumps", "over", "the", "lazy", "dog"]

Bigram	Frequency	Probability
The-quick	1	$1/2 = 0.5$
quick-brown	1	$1/1 = 1$
brown-fox	1	$1/1 = 1$
fox-jumps	1	$1/1 = 1$
jumps-over	1	$1/1 = 1$
over-the	1	$1/1 = 1$
the-lazy	1	$1/2 = 0.5$
lazy-dog	1	$1/1 = 1$

N-grams Model?

Training Data:

Sentence: "The quick brown fox jumps over the lazy dog."

Tokenization: ["The", "quick", "brown", "fox", "jumps", "over", "the", "lazy", "dog"]

Bigram Probability Calculation:

$P(\text{word2} \mid \text{word1}) = \text{Frequency of bigram} / \text{Frequency of word1}$

Example:

$P(\text{quick} \mid \text{The}) = \text{Frequency of "The-quick"} / \text{Frequency of "The"} \\ = 1 / 2 = 0.5$

Bigram	Frequency	Probability
The-quick	1	$1/2 = 0.5$
quick-brown	1	$1/1 = 1$
brown-fox	1	$1/1 = 1$
fox-jumps	1	$1/1 = 1$
jumps-over	1	$1/1 = 1$
over-the	1	$1/1 = 1$
the-lazy	1	$1/2 = 0.5$
lazy-dog	1	$1/1 = 1$

N-grams Model?

Training Data:

Sentence: "The quick brown fox jumps over the lazy dog."

Tokenization: ["The", "quick", "brown", "fox", "jumps", "over", "the", "lazy", "dog"]

Trigram	Frequency	Probability
The-quick-brown	1	$1/1 = 1$
quick-brown-fox	1	$1/1 = 1$
brown-fox-jumps	1	$1/1 = 1$
fox-jumps-over	1	$1/1 = 1$
jumps-over-the	1	$1/1 = 1$
over-the-lazy	1	$1/1 = 1$
the-lazy-dog	1	$1/1 = 1$

N-grams Model?

Training Data:

Sentence: "The quick brown fox jumps over the lazy dog."

Tokenization: ["The", "quick", "brown", "fox", "jumps", "over", "the", "lazy", "dog"]

Trigram Probability Calculation:

$P(\text{word3} \mid \text{word1}, \text{word2}) = \text{Frequency of trigram} / \text{Frequency of bigram}$

Example:

$P(\text{brown} \mid \text{The}, \text{quick}) = \text{Frequency of "The-quick-brown"} / \text{Frequency of "The-quick"}$
 $= 1 / 1 = 1.0$

Trigram	Frequency	Probability
The-quick-brown	1	1/1 = 1
quick-brown-fox	1	1/1 = 1
brown-fox-jumps	1	1/1 = 1
fox-jumps-over	1	1/1 = 1
jumps-over-the	1	1/1 = 1
over-the-lazy	1	1/1 = 1
the-lazy-dog	1	1/1 = 1

N-grams Model?

- An N-gram language model predicts the probability of a given N-gram within any sequence of words in the language.
- A good N-gram model can predict the next word in the sentence i.e the value of $p(w|h)$
- Example of N-gram such as unigram ("This", "article", "is", "on", "NLP") or bi-gram ('This article', 'article is', 'is on', 'on NLP').

Assign a probability to every sentence (or any string of words)

- Finite vocabulary (e.g., words or characters)
- Infinite set of sequences

$$\sum_{e \in \Sigma^*} p_{LM}(e) = 1$$
$$p_{LM}(e) \geq 0, \forall e \in \Sigma^*$$

A Trivial Model

- Assume we have n training sentences
- Let $x_1, x_2, \dots, x_n$ be a sentence, and $c(x_1, x_2, \dots, x_n)$ be the number of times it appeared in the training data.
- Define a language model $p(x_1, x_2, \dots, x_n) = \frac{c(x_1, x_2, \dots, x_n)}{N}$
- No generalization!

Trivial

Will the Sun Rise Tomorrow?

Laplace's Rule of Succession:
On day $n+1$, we've observed that
the sun has risen s times before.



$$p_{Lap}(S_{n+1} = 1 \mid S_1 + \dots + S_n = s) = \frac{s + 1}{n + 2}$$

What's the probability on day 0?

On day 1?

On day 10^6 ?

Start with prior assumption of equal rise/not-rise
probabilities; *update* after every observation.

Smoothing

Smoothing is a technique used to overcome the problem of the sparsity of training data by adding or adjusting the probability mass distribution of words.

➤ Why do we use Smoothing in NLP?

We use Smoothing in NLP to assign non-zero probabilities to unseen words and improve our model accuracy.

Laplace (Add One) Smoothing

- From our earlier example:
 $p_{ML}(\text{beginning} \mid \text{in the}) = 5/100?$ reduce!
 $p_{ML}(\text{end} \mid \text{in the}) = 8/100?$ reduce!
 $p_{ML}(\text{kitchen} \mid \text{in the}) = 0/100?$ increase!

Smoothing

Laplace (Add One) Smoothing

- Let V be the vocabulary size:
i.e., the number of unique words that could follow “in the”

- From our earlier example:

$$p_{ML}(\text{beginning} \mid \text{in the}) = (5 + 1) / (100 + V)$$

$$p_{ML}(\text{end} \mid \text{in the}) = (8 + 1) / (100 + V)$$

$$p_{ML}(\text{kitchen} \mid \text{in the}) = (0 + 1) / (100 + V)$$

Smoothing

Laplace / Add-1 Smoothing

Here, we simply add 1 to all the counts of words so that we never incur 0 value.

$$P_{\text{Laplace}}(w_i | w_{(i-1)}) = (\text{count}(w_i | w_{(i-1)}) + 1) / (\text{count}(w_{(i-1)}) + V)$$

Where V= total words in the training set, 9 in our example.

So, $P(\text{"I like mathematics"})$

$$= P(I | \langle S \rangle) * P(\text{like} | I) * P(\text{mathematics} | \text{like}) * P(\langle E \rangle | \text{mathematics})$$

$$= ((1+1) / (3+9)) * ((1+1) / (1+9)) * ((0+1) / (1+9)) * ((1+1) / (3+9))$$

$$= 1 / 1800$$

Smoothing

Smoothing techniques are used to adjust the probability values calculated by N-Gram models to:

1. Handle zero-probability issues (e.g., unseen word combinations)
2. Reduce overfitting
3. Improve generalization

Common Smoothing techniques:

1. **Laplace Smoothing:** Add 1 to each count (numerator and denominator)

$$P(\text{word}) = (\text{Count}(\text{word}) + 1) / (\text{Total Count} + V)$$

2. **Good-Turing Smoothing:** Estimate probability of unseen events

$$P(\text{word}) = (\text{Count}(\text{word}) / \text{Total Count}) * (1 - (\text{Unseen Count} / \text{Total Count}))$$

3. **Kneser-Ney Smoothing:** Interpolate between N-Gram models

$$P(\text{word}) = \lambda * P(\text{word} \mid \text{context}) + (1-\lambda) * P(\text{word})$$

Smoothing

4. Jelinek-Mercer Smoothing: Linear interpolation

$$P(\text{word}) = \lambda * P(\text{word} \mid \text{context}) + (1-\lambda) * P(\text{word})$$

5. Additive Smoothing: Add a constant to each probability

$$P(\text{word}) = (\text{Count}(\text{word}) + \alpha) / (\text{Total Count} + \alpha * V)$$

In the context of smoothing techniques, α (alpha) and λ (lambda) are hyperparameters that control the amount of smoothing.

Alpha (α)

- Used in:
 - Additive Smoothing
 - Laplace Smoothing
- **Interpretation:** Amount of smoothing applied
- Range: $\alpha \geq 0$
- Effects:
 - $\alpha = 0$: No smoothing (original probabilities)
 - $\alpha > 0$: Smoothing increases (probabilities become more uniform)

Smoothing

Lambda (λ)

- Used in:
 - Jelinek-Mercer Smoothing
 - Kneser-Ney Smoothing
- **Interpretation:** Interpolation weight between models
- Range: $0 \leq \lambda \leq 1$
- **Effects:**
 - $\lambda = 0$: Only consider lower-order model
 - $\lambda = 1$: Only consider higher-order model
 - $0 < \lambda < 1$: Interpolate between models

Hyperparameter Tuning

- Choose α and λ values based on:
 - Cross-validation performance
 - Perplexity score
 - Domain expertise
- Common ranges:
 - α : 0.1 to 1.0
 - λ : 0.5 to 0.9

Thank
you!!!



Lecture-#16

Natural Language Processing

Language modeling

Language Model Evaluation

- ❖ A language model computes the probability of a word sequence W .
- ❖ How can we tell whether a language model is successful at estimating word sequence probabilities?
- ❖ Two criteria are used: **coverage rate** and **perplexity** on a held-out test set that does not form part of the training data.
- ❖ The **coverage rate** measures the percentage of n -grams in the test set that are represented in the language model.
- ❖ **The proportion of input data (e.g., words, phrases, or sentences) that a language model can successfully process or generate.**
- ❖ A special case of this is the out-of-vocabulary rate (or OOV rate), which is 100 minus the unigram coverage rate, or, in other words, the percentage of unique word types *not* covered by the language model.

Language Model Evaluation

❖ Formula:

❖ Coverage Rate = (Number of covered inputs / Total number of inputs) $\times$ 100

❖ Types of Coverage:

1. **Vocabulary Coverage:** Measures the percentage of unique words in the vocabulary that the model can generate.
2. **Sentence Coverage:** Measures the percentage of sentences that the model can generate correctly.
3. **Token Coverage:** Measures the percentage of tokens (words or subwords) that the model can generate.

Language Model Evaluation

Example:

Suppose we have a language model trained on a dataset of movie reviews. We want to evaluate its coverage rate on a test set of 1000 movie reviews.

Vocabulary Size: 10,000 unique words

Test Set:

Review ID	Text
---	---
1	"The movie was amazing!"
2	"I loved the special effects."
3	"The plot was unpredictable."
...	...
1000	"The cinematography was stunning."

Language Model Evaluation

Example:

Suppose we have a language model trained on a dataset of movie reviews. We want to evaluate its coverage rate on a test set of 1000 movie reviews.

Vocabulary Size: 10,000 unique words

Model Output:

Review ID	Generated Text	Covered?
---	---	---
1	"The movie was amazing!"	
2	"I loved the visual effects."	
3	"The plot was surprising."	
...	...	...
1000	"The photography was great."	

Language Model Evaluation



Learning from Experience, Sharing with Passion

Coverage Rate Calculation:

Token Coverage: The model correctly generates 950 out of 1000 reviews' tokens (words or subwords).

$$\text{Token Coverage Rate} = (950/1000) \times 100 = 95\%$$

Sentence Coverage: The model correctly generates 900 out of 1000 reviews' sentences.

$$\text{Sentence Coverage Rate} = (900/1000) \times 100 = 90\%$$

Vocabulary Coverage: The model covers 9,500 out of 10,000 unique words in the vocabulary.

$$\text{Vocabulary Coverage Rate} = (9,500/10,000) \times 100 = 95\%$$

Language Model Evaluation



Learning from Experience, Sharing with Passion

Analysis:

- The model excels in token coverage (95%) and vocabulary coverage (95%).
- Sentence coverage (90%) is slightly lower, indicating some issues with sentence-level generation.
- The model struggles with words like "cinematography" (Review 1000), which is an out-of-vocabulary (OOV) word.

Improvement Strategies:

1. Increase vocabulary size to include more rare words.
2. Use subword modeling to handle OOV words.
3. Fine-tune the model on a dataset with more diverse sentence structures.

Coverage Rate Thresholds:

- Token Coverage: Excellent (95%)
- Sentence Coverage: High (90%)
- Vocabulary Coverage: Excellent (95%)

Language Model Evaluation



Learning from Experience, Sharing with Passion

Perplexity Score Calculation:

Definition: Perplexity measures the uncertainty or confusion of a language model when predicting the next word in a sequence.

Formula:

$$\text{Perplexity (PPL)} = 2^{(-\text{entropy})}$$

where entropy is the average negative log-likelihood of the model's predictions.

Entropy Calculation:

$$\text{Entropy} = - (1/N) * \sum(\log_2(p(w_i)))$$

where:

- N is the total number of words
- $p(w_i)$ is the predicted probability of word w_i
- $\log_2$ is the logarithm to the base 2

Language Model Evaluation

- ❖ The second criterion, perplexity, is an information theoretic measure.
- ❖ Given a model p of a discrete probability distribution, perplexity can be defined as 2 raised to the entropy of p :

$$PPL(p) = 2^{H(p)} = 2^{-\sum_x p(x) \log_2 p(x)}$$

- ❖ In language modeling, we are often more interested in the performance of a language model q on a test set of a fixed size, say t words ($w_1 w_2 \dots w_t$). Then, language model perplexity can be computed as

$$PPL(p, q) = 2^{H(p, q)} = 2^{-\sum_{i=1}^t p(w_i) \log_2 q(w_i)}$$

$$2^{-\frac{1}{t} \sum_{i=1}^t \log_2 q(w_i)}$$

Language Model Evaluation

❖ where $q(w_i)$ computes the probability of the i 'th word. If $q(w_i)$ is an n -gram probability, the equation becomes

$$2^{-\frac{1}{T} \sum_{i=1}^T \log_2 p(w_i | w_{i-1}, \dots, w_{i-n+1})}$$

❖ When comparing different language models, especially models based on different ways of decomposing a text into language modeling units (e.g., words vs. morphemes), care must be taken to normalize their perplexities with respect to the same number of units in order to obtain a meaningful comparison.

Language Model Evaluation

- ❖ **Perplexity** can be thought of as the average number of equally likely successor words when transitioning from one position in the word string to the next.
- ❖ If the model has no predictive power at all, perplexity is equal to the vocabulary size.
- ❖ A model achieving perfect prediction, by contrast, has a perplexity of one.
- ❖ The goal in language model development is most often to minimize the perplexity on a held-out data set representative of the domain of interest.

Language Model Evaluation

Suppose we have a language model that predicts the following probabilities for a 5-word sequence:

Word	Predicted Probability
------	-----------------------

w1	0.4
w2	0.3
w3	0.2
w4	0.05
w5	0.05

- w1: 40% chance of being the next word
- w2: 30% chance of being the next word
- w3: 20% chance of being the next word
- w4 and w5: 5% chance each of being the next word

$$\text{Entropy} = - (1/5) * (\log_2(0.4) + \log_2(0.3) + \log_2(0.2) + \log_2(0.05) + \log_2(0.05))$$
$$= 2.805$$

$$\text{Perplexity (PPL)} = 2^{(-2.805)} \approx 0.143$$

Language Model Evaluation

With a perplexity score of approximately 0.143, here's the interpretation:

Perplexity Score: 0.143

Interpretation:

- **Extremely low perplexity:** Your model is exceptionally confident in its predictions.
- **Near-perfect performance:** The model is almost perfectly predicting the next word.

Implications:

- **Overfitting:** The model might be overly specialized to the training data.
- **Data leakage:** The test data might be identical to or very similar to the training data.

Perplexity Score Categories:

- Ideal: 1 (perfect prediction)
- Excellent: 0.1-1 (near-perfect prediction)
- Good: 1-5 (reasonable performance)
- Fair: 5-10 (moderate performance)
- Poor: 10-50 (poor performance)
- Very Poor: 50+ (extremely poor performance)

Parameter Estimation

Maximum-Likelihood Estimation and Smoothing

- ❖ The standard procedure in training n -gram models is to estimate n -gram probabilities using the maximum-likelihood criterion in combination with parameter smoothing.
- ❖ The maximum likelihood estimate is obtained by simply computing relative frequencies:

$$P(w_i | w_{i-1}, w_{i-2}) = \frac{c(w_i, w_{i-1}, w_{i-2})}{c(w_{i-1}, w_{i-2})}$$

- ❖ where $c(w_i, w_{i-1}, w_{i-2})$ is the count of the trigram $w_{i-2}w_{i-1}w_i$ in the training data.
- ❖ It is obvious that this method fails to assign nonzero probabilities to word sequences that have not been observed in the training data; on the other hand, the probability of sequences that were observed might be overestimated.

Maximum Likelihood Estimation

Maximum Likelihood Estimation

Unigram Language Model θ

$$p(w|\theta)=?$$

10/100

5/100

3/100

3/100

1/100

...
text ?
mining ?
association ?
database ?
...
query ?
...

Estimation

Document

text 10
mining 5
association 3
database 3
algorithm 2
...
query 1
efficient 1

A paper (total #words=100)

Maximum Likelihood Estimation



Learning from Experience, Sharing with Passion

Definition: Maximum Likelihood Estimation is a statistical method for estimating the parameters of a probabilistic model. It identifies the parameter values that maximize the likelihood function, which measures how likely the observed data is, given a set of parameters.

Formula:

For a dataset $X=\{x_1, x_2, \dots, x_n\}$ and a probability model $P(x|\theta)$, where θ represents the model's parameters, the likelihood is:

$$L(\theta; X) = P(X|\theta) = \prod_{i=1}^n P(x_i|\theta)$$

The log-likelihood, often used for computational simplicity, is:

$$\ell(\theta; X) = \log L(\theta; X) = \sum_{i=1}^n \log P(x_i|\theta)$$

Maximum Likelihood Estimation

To find the MLE, maximize $\ell(\theta; X)$

$$\hat{\theta}_{MLE} = \arg \max_{\theta} \ell(\theta; X)$$

Example:

For a coin toss, if $x_i \in \{0,1\}$ represents the outcome (0: tails, 1: heads), and the probability of heads is θ , then:

$$P(x_i|\theta) = \theta^{x_i} (1 - \theta)^{1-x_i}$$

After tossing the coin multiple times:

Count the number of heads.

$$\hat{\theta} = \frac{\text{Number of Heads}}{\text{Total Tosses}}$$

The MLE for θ is :

Maximum Likelihood Estimation

Maximum Likelihood Estimation

- For N-gram language models

- $$p(w_i | w_{i-1}, \dots, w_{i-n+1}) = \frac{c(w_i, w_{i-1}, \dots, w_{i-n+1})}{c(w_{i-1}, \dots, w_{i-n+1})}$$

Smoothing

Definition: Smoothing adjusts probabilities to avoid overfitting, especially for events with zero frequency in the observed data. It's often used in language models, where unseen words or n-grams may have a zero probability under MLE.

Common Smoothing Techniques:

1.Additive Smoothing (Laplace Smoothing): Adds a small constant α to each count:

$$P(w|c) = \frac{\text{Count}(w, c) + \alpha}{\text{Total Count}(c) + \alpha \cdot V}$$

V: Vocabulary Size

$\alpha = 1$, Standard Laplace Smoothing

Smoothing

2. Good-Turing Smoothing: Adjusts probabilities based on the frequency of frequencies:

$$P(w) = \frac{(N_{k+1})}{N \cdot N_k}$$

- N_k : Number of events that occur k times
- N : Total number of events

Smoothing

3. Interpolation: Combines probabilities from different models (e.g., unigram, bigram, trigram):

$$P(w|h) = \lambda_1 P_{unigram}(w) + \lambda_2 P_{bigram}(w|h) + \lambda_3 P_{trigram}(w|h_2 h_1)$$

λ_i : Weights summing to 1

Smoothing

4. Backoff: Uses lower-order models when higher-order probabilities are zero:

$$P(w|h) = \begin{cases} P_{trigram}(w|h_2h_1) & \text{if count} > 0 \\ \alpha(h)P_{bigram}(w|h_1) & \text{otherwise} \end{cases}$$

$\alpha(h)$: Normalization factor

Smoothing

Example:

In a bigram model, if the bigram "NewYork" appears 5 times and the unigram "New" appears 10 times, MLE gives:

$$P("York"|"New") = \frac{5}{10} = 0.5$$

If "NewTokyo" is unseen, MLE assigns $P("Tokyo"|"New") = 0$.

With Laplace smoothing ($\alpha = 1$ and $V = 1000$):

$$P("Tokyo"|"New") = \frac{0 + 1}{10 + 1000} \approx 0.00091$$

Maximum Likelihood Estimation and Smoothing



Learning from Experience, Sharing with Passion

- **MLE** often produces exact probabilities based on observed data, which may lead to zero probabilities for unseen events.
- **Smoothing** techniques adjust these probabilities, ensuring better generalization for unseen events in applications like language modeling, recommendation systems, and machine translation.

Backoff Smoothing

- Backoff is a smoothing technique used in probabilistic language models, particularly for handling cases where higher-order n-grams (e.g., trigrams) have zero frequency in the training data.
- The idea is to "back off" to lower-order models (e.g., bigrams or unigrams) that are more general but still informative.
- **Hierarchy of Models:**
 - Higher-order n-grams (e.g., trigrams) provide more context but are less likely to have been observed frequently in the training data.
 - Lower-order n-grams (e.g., bigrams or unigrams) are more likely to be observed but offer less contextual specificity.
- **Probability Adjustment:**
 - If a higher-order n-gram's probability is zero, use a lower-order n-gram.
 - To ensure a valid probability distribution, the probabilities are scaled using a **normalization factor**.
- **Flexibility:**
 - Backoff assigns a probability even when certain n-grams are unseen, preventing zero probabilities in applications like speech recognition or text prediction.

Backoff Smoothing

- For an n-gram model, the probability of a word w_i its context $h = w_{i-n+1}^{i-1}$

$$P(w_i|h) = \begin{cases} P_{n-gram}(w_i|h) & \text{if Count}(w_i, h) > 0 \\ \alpha(h) \cdot P_{lower}(w_i|h') & \text{otherwise} \end{cases}$$

Where:

- $P_{n-gram}(w_i|h)$: Higher-order n-gram probability, e.g., $P(w_i|w_{i-2}w_{i-1})$ for trigrams.
- $P_{lower}(w_i|h')$: Probability from a lower-order n-gram, e.g., $P(w_i|w_{i-1})$ for bigrams.
- $\alpha(h)$: Normalization factor ensuring that the probabilities sum to 1.

Backoff Smoothing

- The normalization factor is:

$$\alpha(h) = \frac{1 - \sum_{w \in W_h} P_{n-gram}(w|h)}{\sum_{w \in W_h} P_{lower}(w|h')}$$

W_h : Words w that have a zero count in the context h .

Backoff Smoothing

Steps in Backoff Smoothing

1. Check Higher-Order n-Gram:

- If the count of a higher-order n-gram (e.g., $P(w|w_{i-2}, w_{i-1})$) is greater than zero, use its probability.

2. Fallback to Lower-Order n-Gram:

- If the higher-order n-gram's count is zero, use the probability from the lower-order model (e.g., $P(w|w_{i-1})$).

3. Normalize Probabilities:

- Use the normalization factor $\alpha(h)$ to scale the lower-order probabilities.

4. Iterate:

- Repeat the process, backing off to unigrams if necessary.

Backoff Smoothing

Example

Suppose we are working with a trigram model, and the vocabulary includes the words: {"I", "like", "coffee", "tea"}.

• Training data:

- “I like”: 5 occurrences
- “likecoffee” : 2 occurrences
- “liketea” : 1 occurrence
- “Icoffee”: 0 occurrences (unseen trigram)

Backoff Smoothing

Backoff Process

1. Trigram:

- For $P(\text{"coffee"} | \text{"Ilike"})$: Use $\frac{\text{Count}(\text{"Ilikecoffee"})}{\text{Count}(\text{"Ilike"})} = \frac{2}{5} = 0.4$.
- For $P(\text{"coffee"} | \text{"Icoffee"})$: Unseen, back off to bigram.

2. Bigram:

- $P(\text{"coffee"} | \text{"coffee"})$: Unseen, back off to unigram.

3. Unigram:

- $P(\text{"coffee"})$: Use unigram probability: $\frac{\text{Count}(\text{"coffee"})}{\text{Total Words}}$.

Thank
you!!!



Lecture-#17

Natural Language Processing

Language modeling

Bayesian Parameter Estimation

- ❖ Bayesian probability estimation is an alternative parameter estimation method whereby the set of parameters of a model is itself viewed as a random variable governed by a prior statistical distribution.
- ❖ Given a training sample S and a set of parameters θ , $P(\theta)$ denotes a prior distribution over different possible values of θ , and $P(\theta | S)$ is the posterior distribution, which can be expressed using Bayes's rule as

$$P(\theta | S) = \frac{P(S | \theta)P(\theta)}{P(S)} \quad (5.16)$$

- ❖ In language modeling, the set of parameters is the vector of word probabilities, that is, $\theta = \langle P(w_1), \dots, P(w_K) \rangle$ (where K is the vocabulary size) for a unigram model, or, more generally, $\theta = \langle P(w_1 | h_1), \dots, P(w_K | h_K) \rangle$ for an n -gram model with K n -grams and history h of a specified length.

Bayesian Parameter Estimation

- ❖ The training sample S is a sequence of words, $w_1 \dots w_t$. We require a point estimate of θ given the constraints expressed by the prior distribution and the training sample.
- ❖ This can be done using either the **maximum a posteriori (MAP)** criterion or the Bayesian criterion.

$$\theta^{MAP} = \underset{\theta \in \Theta}{\operatorname{argmax}} P(\theta|S) = \underset{\theta \in \Theta}{\operatorname{argmax}} P(S|\theta)P(\theta) \quad (5.17)$$

- ❖ where Θ is the space of all possible assignments for θ . The Bayesian criterion finds the *expected* value of θ given the sample S :

- ❖
$$\theta^B = E[\theta|S] = \int_{\Theta} \theta P(\theta|S) d\theta \quad (5.18)$$

$$= \frac{\int_{\Theta} \theta P(S|\theta)P(\theta) d\theta}{\int_{\Theta} P(S|\theta)P(\theta) d\theta} \quad (5.19)$$

Bayesian Parameter Estimation

- $\Pr(\text{sick}|I) = 0.001$ so $\Pr(\text{healthy}|I) = 0.999$.
- $\Pr(\text{neg}|\text{sick}, I) = 0.01$ so $\Pr(\text{pos}|\text{sick}, I) = 0.99$.
- $\Pr(\text{pos}|\text{healthy}, I) = 0.02$ so $\Pr(\text{neg}|\text{healthy}, I) = 0.98$.

Additionally, since $\text{pos} = (\text{pos}, \text{sick}) \vee (\text{pos}, \text{healthy})$,

$$\begin{aligned}\Pr(\text{pos}|I) &= \Pr(\text{pos}, \text{sick}|I) + \Pr(\text{pos}, \text{healthy}|I) \\ &= \Pr(\text{pos}|\text{sick}, I) \Pr(\text{sick}|I) \\ &\quad + \Pr(\text{pos}|\text{healthy}, I) \Pr(\text{healthy}|I) \\ &= 0.99 \times 0.001 + 0.02 \times 0.999 = 0.00099 + 0.01998 \\ &= 0.02097\end{aligned}\tag{1.4}$$

We can then use Bayes's theorem to show that

$$\Pr(\text{sick}|\text{pos}, I) = \frac{0.00099}{0.02097} \approx 0.04721\tag{1.5}$$

I.e., if I test positive for TDY, I have about a 4.7% chance of actually having the disease. This is a lot less than $\Pr(\text{pos}|\text{sick}, I)$, which is 99%!

Bayesian Parameter Estimation

	Positive	Negative	Total
Sick	990	10	1,000
Healthy	19,980	979,020	999,000
Total	20,970	979,030	1,000,000

(Of course, if we choose a *sample* of a million individuals out of a larger population, we won't expect to get exactly this number of results, but the representative population is still useful conceptual construct.)

Translating from numbers in this hypothetical population, we can confirm that it captures the input information:

$$P(\text{sick}) = \frac{1,000}{1,000,000} = .001 \quad (1.6a)$$

$$P(\text{positive}|\text{healthy}) = \frac{19,980}{999,000} = .02 \quad (1.6b)$$

$$P(\text{negative}|\text{sick}) = \frac{10}{1,000} = .01 \quad (1.6c)$$

But now we can also calculate what we want, the conditional probability of being sick given a positive result. That is the fraction of the total number of individuals with positive test results that are in the “sick *and* positive” category:

$$P(\text{sick}|\text{positive}) = \frac{990}{20,970} \approx .04721 \quad (1.7)$$

or about 4.7%, as before.

Bayesian Parameter Estimation

- In language modeling, Bayesian parameter estimation is used to predict the likelihood of word sequences by combining prior beliefs about language patterns with observed data. This approach provides a principled way to handle sparse data and model uncertainty, which are common issues in natural language processing (NLP).

Bayesian Framework for Language Models

1. Objective:

Estimate the probability $P(w|h)$, where w is the next word, and h is the context (e.g., a sequence of preceding words).

2. Model Parameters:

The parameters θ represent the probabilities of word sequences, such as $\theta(w|h)=P(w|h)$.

Bayesian Parameter Estimation

3. Bayesian Estimation:

Using Bayes' theorem, the posterior distribution of θ is:

$$P(\theta|D) = \frac{P(D|\theta)P(\theta)}{P(D)}$$

Where:

- **P(θ)**: Prior distribution, representing initial beliefs about θ .
- **P($D|\theta$)**: Likelihood, representing the probability of the observed data D given θ .
- **P(D)**: Evidence (normalizing constant).

Bayesian Parameter Estimation

1. Define the Prior Distribution:

The prior reflects assumptions about word probabilities before observing data. Common choices include:

- **Dirichlet Prior:** For multinomial distributions like n-grams, a Dirichlet prior is used because it is conjugate to the multinomial likelihood. If α is the concentration parameter, the prior is:

$$P(\theta) \propto \prod_{w,h} \theta(w|h)^{\alpha_{w,h}-1}$$

- Larger α : Stronger prior beliefs (smoother estimates).
- Smaller α : Weaker prior beliefs (more data-driven).

Bayesian Parameter Estimation

2. Define the Likelihood:

The likelihood is based on the observed n-gram counts in the data D :

$$P(D|\theta) = \prod_{w,h} \theta(w|h)^{C(w,h)}$$

Where $C(w, h)$ is the count of w following h .

3. Compute the Posterior:

Combine the prior and likelihood:

$$P(\theta|D) \propto P(D|\theta)P(\theta)$$

For a Dirichlet prior and multinomial likelihood, the posterior is also a Dirichlet distribution:

$$P(\theta|D) = \text{Dirichlet}(\alpha + C)$$

Bayesian Parameter Estimation

4. Use the Posterior for Predictions:

The probability of the next word is the expectation under the posterior:

$$P(w|h, D) = E[\theta(w|h)|D] = \frac{\alpha_{w,h} + C(w, h)}{\sum_{w'} (\alpha_{w',h} + C(w', h))}$$

Advantages Over Maximum Likelihood Estimation (MLE)

- **Handles Sparse Data:** MLE assigns zero probability to unseen n-grams, while Bayesian estimation smooths probabilities through the prior.
- **Incorporates Prior Knowledge:** Domain knowledge or pre-trained models can be incorporated into the prior.
- **Quantifies Uncertainty:** The posterior captures the uncertainty of parameter estimates, which is valuable for applications requiring confidence scores.

Bayesian Parameter Estimation

Suppose the observed text corpus contains the following sentences:

- **Sentence 1:** "I like tea"
- **Sentence 2:** "I like coffee"

From this corpus, we extract bigram counts:

Bigram	Count $C(w_i, w_{i-1})$
("I", "START")	2
("like", "I")	2
("tea", "like")	1
("coffee", "like")	1

The vocabulary V includes {"I", "like", "tea", "coffee", "START", "END"}

Bayesian Parameter Estimation

For context $h = \text{"like"}$:

- Vocabulary: $\{\text{"tea"}, \text{"coffee"}, \text{"I"}, \text{"START"}, \text{"END"}\}$.
- Bigram counts $C(w, h)$:
 - $C(\text{"tea"}, \text{"like"}) = 1$
 - $C(\text{"coffee"}, \text{"like"}) = 1$
 - $C(w, \text{"like"}) = 0$ for other words.
- Prior: $\alpha = 1$ for all words.

Posterior Parameters:

$$\alpha_{\text{tea}, \text{like}}^{\text{posterior}} = 1 + 1 = 2$$

$$\alpha_{\text{coffee}, \text{like}}^{\text{posterior}} = 1 + 1 = 2$$

$$\alpha_{\text{I}, \text{like}}^{\text{posterior}} = \alpha_{\text{START}, \text{like}}^{\text{posterior}} = \alpha_{\text{END}, \text{like}}^{\text{posterior}} = 1$$

Bayesian Parameter Estimation

Predictive Probabilities:

$$P(w|h = \text{"like"}) = \frac{\alpha_{w,like}^{\text{posterior}}}{\sum_{w'} \alpha_{w',like}^{\text{posterior}}}$$

$$P(\text{"tea"}|\text{"like"}) = \frac{2}{2 + 2 + 1 + 1 + 1} = \frac{2}{7} \approx 0.286$$

$$P(\text{"coffee"}|\text{"like"}) = \frac{2}{7} \approx 0.286$$

$$P(\text{"I"}|\text{"like"}) = \frac{1}{7} \approx 0.143$$

$$P(\text{"START"}|\text{"like"}) = P(\text{"END"}|\text{"like"}) = \frac{1}{7} \approx 0.143$$

Bayesian Parameter Estimation



Learning from Experience, Sharing with Passion

The Bayesian approach assigns non-zero probabilities to all words in the vocabulary, even if they were not observed with a given context in the training data.

For example:

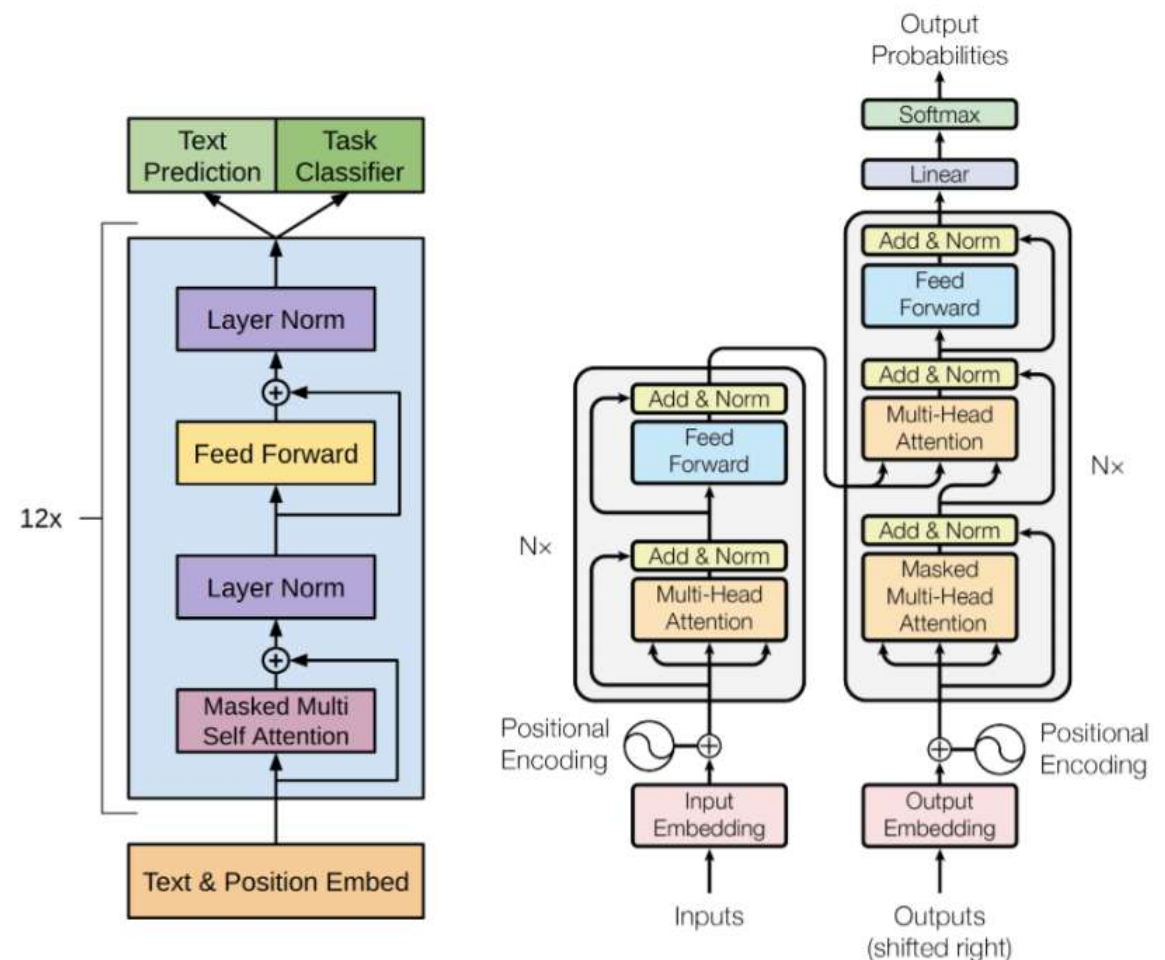
- $P("I" \mid "like") > 0$ even though ("I", "like") was not observed.

This smoothing effect ensures robustness in language modeling, especially for unseen word sequences.

Large Language Model

What is a Large Language Model?

A large language model is an advanced type of language model that is trained using deep learning techniques on massive amounts of text data. These models are capable of generating human-like text and performing various natural language processing tasks.



Large Language Model



Learning from Experience, Sharing with Passion

How do you train an LLM from scratch?

A. Training a Large Language Model (LLM) from scratch is a complex and resource-intensive process. It involves several key steps:

1. **Data Collection:** Gather a vast and diverse text corpus from the internet or other sources. The quality and quantity of data are crucial.
2. **Data Preprocessing:** Clean and format the data, including tokenization (breaking text into words or subword units) and handling special characters.
3. **Model Architecture:** Choose an architecture like a Transformer-based model and design its structure, including the number of layers, attention mechanisms, and other hyperparameters.
4. **Initialization:** Initialize model parameters with random values or pre-trained weights from a similar model.
5. **Training Objective:** Define a training objective, typically maximum likelihood estimation (MLE) or other variants.
6. **Training:** Use backpropagation and optimization algorithms (e.g., Adam) to update model parameters, minimizing the chosen loss function.
7. **Regularization:** Apply techniques like dropout and layer normalization to prevent overfitting.
8. **Hyperparameter Tuning:** Fine-tune various hyperparameters to optimize model performance.
9. **Evaluation:** Assess the model's performance using appropriate metrics and validation datasets.
10. **Inference:** Once trained, the LLM can be used for various natural language processing tasks, such as text generation, translation, and summarization.

Large Language Model Types



Learning from Experience, Sharing with Passion

- ❖ **GPT-3 (Generative Pre-trained Transformer 3)** – This is one of the largest Large Language Models developed by [OpenAI](#). It has 175 billion parameters and can perform many tasks, including text generation, translation, and summarization.
- ❖ **BERT (Bidirectional Encoder Representations from Transformers)** – Developed by Google, BERT is another popular LLM that has been trained on a massive corpus of text data. It can understand the context of a sentence and generate meaningful responses to questions.
- ❖ **XLNet** – This LLM developed by Carnegie Mellon University and Google uses a novel approach to language modeling called “permutation language modeling.” It has achieved state-of-the-art performance on language tasks, including language generation and question answering.
- ❖ **T5 (Text-to-Text Transfer Transformer)** – T5, developed by Google, is trained on a variety of language tasks and can perform text-to-text transformations, like translating text to another language, creating a summary, and question answering.
- ❖ **RoBERTa (Robustly Optimized BERT Pretraining Approach)** – Developed by Facebook AI Research, RoBERTa is an improved BERT version that performs better on several language tasks.

Types of Language Model

Class-Based Language Models

- ❖ Class-based language models are a simple way of addressing data sparsity in language modeling.
- ❖ Words are first clustered into classes, either by automatic means or based on linguistic criteria, for example, using part-of-speech (POS) classes.
- ❖ The statistical model makes the assumption that words are conditionally independent of other words given the current word class.
- ❖ If c_i is the class of word w_i , a class-based bigram model can be defined as follows:

$$P(w_i|w_{i-1}) = \sum_{c_i, c_{i-1}} P(w_i|c_i)P(c_i|c_{i-1}, w_{i-1})P(c_{i-1}|w_{i-1}) \quad (5.29)$$

$$= \sum_{c_i, c_{i-1}} P(w_i|c_i)P(c_i|c_{i-1})P(c_{i-1}|w_{i-1}) \quad (5.30)$$

Types of Language Model

Class-Based Language Models

- ❖ under the assumption that c_i is independent of w_{i-1} given c_{i-1} .
- ❖ Usually, a class will contain more than one word, such that the model simplifies to

$$P(w_i|w_{i-1}) = P(w_i|c_i)P(c_i|c_{i-1}) \quad (5.31)$$

Goodman compared this decomposition to the following model:

$$P(w_i|w_{i-1}) \approx P(w_i|c_i, c_{i-1})P(c_i|c_{i-1}) \quad (5.32)$$

where the current word is conditioned not only on the current word class but also on the preceding word classes.

Types of Language Model

Class-Based Language Models

- Example:

Suppose we have the sentence:

- "I like apples"

Assume the word classes are:

Pronouns (PRON): {I, you, he, she}

Verbs (VERB): {like, love, eat}

Nouns (NOUN): {apples, bananas, oranges}

The class sequence for the sentence is:

C=PRON,VERB,NOUN

Types of Language Model

Class-Based Language Models

- Using the bigram class-based language model formula:

$$P(W) = \prod_{i=1}^n P(w_i | c_i) \times P(c_i | c_{i-1})$$

We calculate:

- $P(\text{PRON} | \text{<start>})P(\text{PRON} | \text{<start>})$: probability that the first class is PRON
- $P(\text{I} | \text{PRON})P(\text{I} | \text{PRON})$: probability of word "I" given class PRON
- $P(\text{VERB} | \text{PRON})P(\text{VERB} | \text{PRON})$: probability of VERB class following PRON
- $P(\text{like} | \text{VERB})P(\text{like} | \text{VERB})$: probability of word "like" given class VERB
- $P(\text{NOUN} | \text{VERB})P(\text{NOUN} | \text{VERB})$: probability of NOUN class following VERB
- $P(\text{apples} | \text{NOUN})P(\text{apples} | \text{NOUN})$: probability of word "apples" given class NOUN

Types of Language Model

Class-Based Language Models

The total probability of the sentence:

$$P(\text{"I like apples"}) = P(\text{PRON} | \text{<start>}) \times P(\text{I} | \text{PRON}) \times P(\text{VERB} | \text{PRON}) \times P(\text{like} | \text{VERB}) \times P(\text{NOUN} | \text{VERB}) \times P(\text{apples} | \text{NOUN})$$

Types of Language Model

Class-Based Language Models

Given Probabilities (example values):

- $P(\text{PRON}|\text{<start>})=0.5$ (Probability that the first word belongs to PRON class)
- $P(\text{I}|\text{PRON})=0.4$ (Probability of word "I" within PRON class)
- $P(\text{VERB}|\text{PRON})=0.6$ (Probability that VERB follows PRON)
- $P(\text{like}|\text{VERB})=0.3$ (Probability of "like" within VERB class)
- $P(\text{NOUN}|\text{VERB})=0.7$ (Probability that NOUN follows VERB)
- $P(\text{apples}|\text{NOUN})=0.5$ (Probability of "apples" within NOUN class)

Types of Language Model

Class-Based Language Models

Step-by-step Calculation:

$$P(\text{"I like apples"}) = P(\text{PRON} | \langle \text{start} \rangle) \times P(\text{I} | \text{PRON}) \times P(\text{VERB} | \text{PRON}) \times P(\text{like} | \text{VERB}) \times P(\text{NOUN} | \text{VERB}) \times P(\text{apples} | \text{NOUN})$$

$$= 0.5 \times 0.4 \times 0.6 \times 0.3 \times 0.7 \times 0.5$$

Final probability:

$$P(\text{"I like apples"}) = 0.0126$$

Types of Language Model

Class-Based Language Models

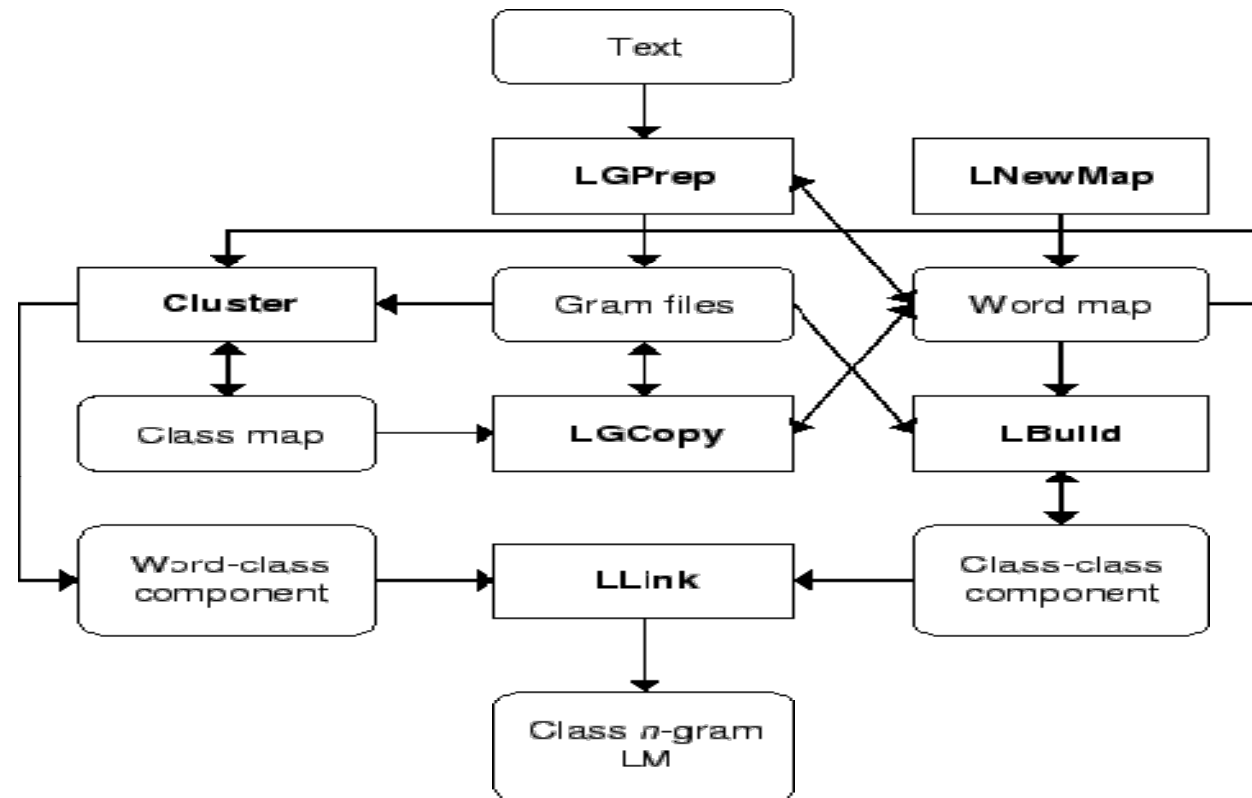


Fig. 14.2 The main stages in building a class-based language model

Types of Language Model

MaxEnt Language Models

- ❖ MaxEnt models represent an alternative where these constraints are relaxed.
- ❖ Rather than setting the probability of a given n -gram to its relative frequency in the training data (modulo smoothing), the requirement of MaxEnt modeling is that the model agree, on average, with the observed counts of events in the training data.
- ❖ The MaxEnt model is formulated as follows:

$$P(y|x) = \frac{1}{Z(x)} \exp \left(\sum_k \lambda_k f_k(x, y) \right) \quad (5.37)$$

where $f(x, y)$ is a feature function defined both on the input and the predicted variables, λ is a feature function specific weight, and $Z(x)$ is a normalization factor.

Types of Language Model

Max Ent Language Models

- ❖ Maximum Entropy (MaxEnt) language models are linear models which are typically regularized using the L1 or L2 terms in the likelihood objective. This obviates the need for smoothed n-gram language models.
- ❖ The [maximum entropy](#) (maxent) approach is rooted in information theory and has been successfully applied to many fields including physics and natural language processing.
- ❖ It creates a model that best accounts for the available data but with a constraint that without any additional information the model should maximize entropy.
- ❖ In other words, the model prefers a uniform distribution by maximizing the [conditional entropy](#).
- ❖ Let $\mathbf{X}$ be the $D \times N$ matrix of training data, as before, so that $\mathbf{x}_j$ is a column vector formed by the j th column of $\mathbf{X}$. Let $\mathbf{W}$ be the $L \times D$ weight matrix, with $\mathbf{w}_i$ the row vector formed by the i th row of $\mathbf{W}$. The probability of category label i given the data $\mathbf{x}_j$ is

Types of Language Model

Max Ent Language Models

$$p_{i,j} = \frac{1}{Z_j} \exp w_i x_j . \quad (19.1)$$

The constant Z_j is the sum of the exponentiated inner products for all the category labels,

$$Z_j = \sum_{i=1}^L \exp (w_i x_j), \quad (19.2)$$

so that $\mathbf{p}_j$ represents a properly normalized probability distribution over labels with $\sum_{i=1}^L p_{i,j} = 1$.

The training examples must have category labels assigned to the data. Let $\mathbf{y}$ be a $N \times 1$ column vector with entries $y_j \in \{1, \dots, L\}$. With all of this, the objective function is the conditional log-likelihood of the labels given the observed data,

$$\mathcal{L}(W; y, X) \equiv \sum_{j=1}^N \log p_{y_j, j} \quad (19.3)$$

Types of Language Model

Max Ent Language Models

Feature Function $f_j(m,c)$	Description
$f_1(m,c) = 1$ if current word ends with "ing" and $c=Verb$, else 0	Detects if word looks like a verb
$f_2(m,c) = 1$ if previous word is "I" and $c=Verb$, else 0	Checks if verb follows "I"

Suppose you're trying to classify a word's part of speech (POS) given the context m . Possible classes for the POS are:

- $c_1 = \text{Noun}$
- $c_2 = \text{Verb}$

Types of Language Model

Max Ent Language Models

Weights: $\lambda_1=0.8$, $\lambda_2=1.2$

Given input:

- Current word: "running" (ends with "ing")
- Previous word: "I"

Step 1: Calculate unnormalized scores for each class

For Verb (c2):

$$\sum_{j=1}^2 \lambda_j f_j(m, \text{Verb}) = 0.8 \times 1 + 1.2 \times 1 = 2.0$$

Types of Language Model

Max Ent Language Models

For Noun (c1):

$$\sum_{j=1}^2 \lambda_j f_j(m, \text{Noun}) = 0.8 \times 0 + 1.2 \times 0 = 0$$

Step 2: Calculate exponentials (unnormalized scores)

$$\exp(2.0)=7.39, \exp(0)=1$$

Step 3: Calculate normalization factor $Z(m)$:

$$Z(m)=7.39+1=8.39$$

Types of Language Model

Max Ent Language Models

Step 4: Calculate probabilities

$$P(\text{Verb}|m)=7.39 / 8.39=0.88$$

$$P(\text{Noun}|m)=1 / 8.39 =0.12$$

- Given the context ("I running"), the model predicts "Verb" with 88% probability and "Noun" with 12%.
- The features and weights influence the prediction, and normalization ensures probabilities sum to 1.

Types of Language Model

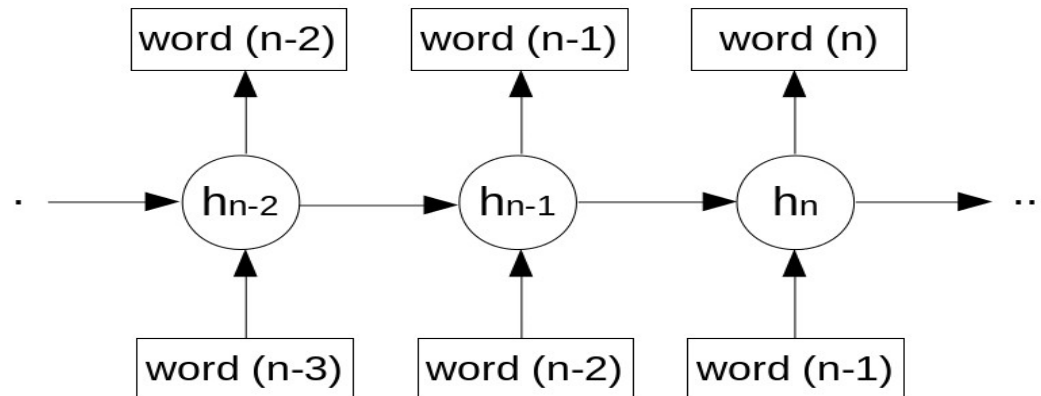
Neural network language model :

- ❖ A neural network language model is a language model based on Neural Networks , exploiting their ability to learn distributed representations to reduce the impact of the *curse of dimensionality*.
- ❖ In the context of learning algorithms, the curse of dimensionality refers to the need for huge numbers of training examples when learning highly complex functions. When the number of input variables increases, the number of required examples can grow exponentially.
- ❖ The curse of dimensionality arises when a huge number of different combinations of values of the input variables must be discriminated from each other, and the learning algorithm needs at least one example per relevant combination of values.
- ❖ In the context of language models, the problem comes from the huge number of possible sequences of words, e.g., with a sequence of 10 words taken from a vocabulary of 100,000 there are 10^5 possible sequences

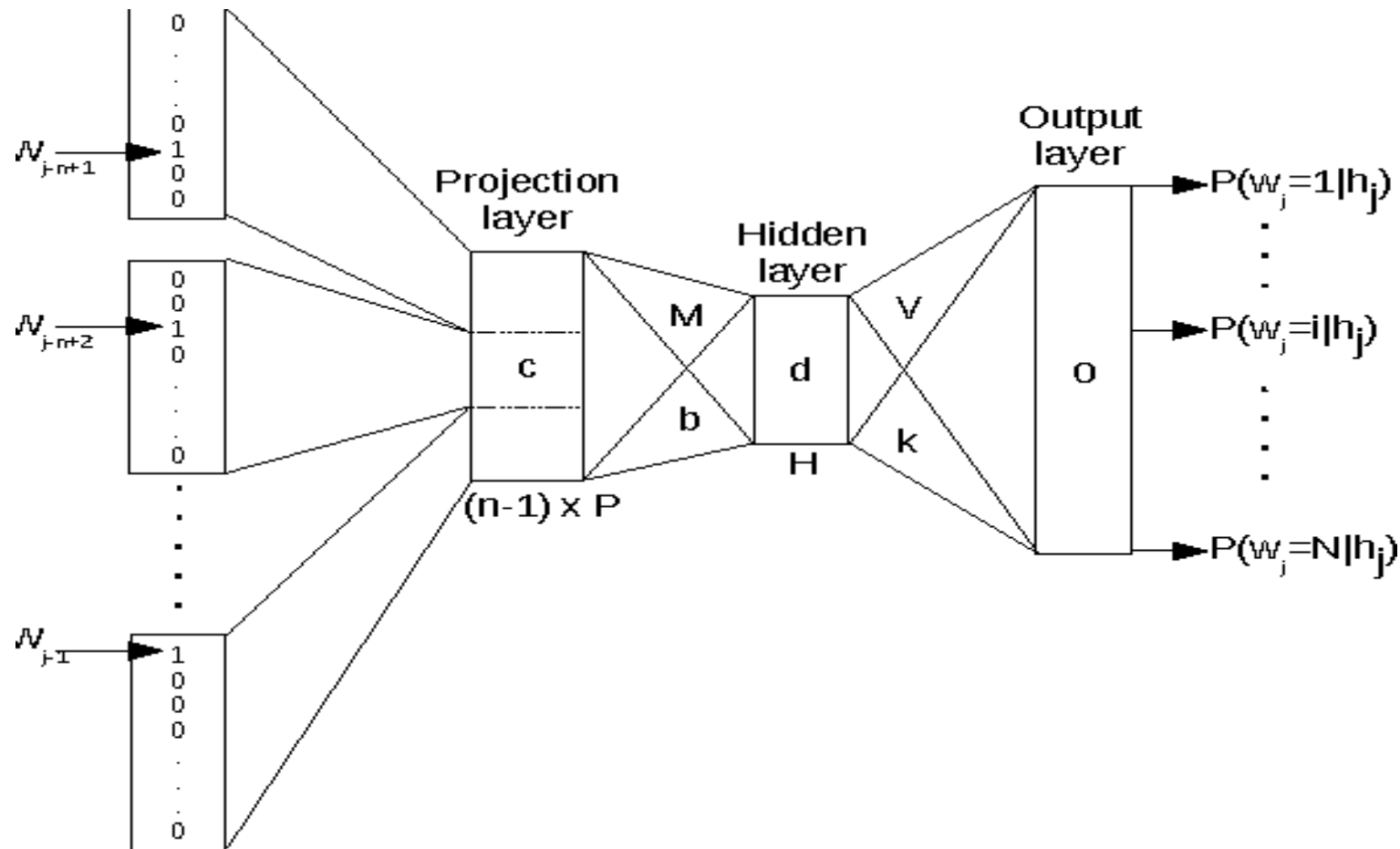
Types of Language Model

Neural network language model :

- ❖ Neural language models based on feed-forward neural networks have a limitation that the input dimension is fixed.
- ❖ Still, many attempts to obtain such statistical models involve approaches that are very specific for language domain – for example, assumption that natural language sentences can be described by parse trees, or that we need to consider morphology of words, syntax and semantics.



Types of Language Model



Types of Language Model

NNLM has following sets of parameters .

1. The matrix which creates the embedding for each word in the context. This distributed embedding is in $\mathbb{R}^m$ space. This matrix is $C: V \times m$
2. The matrix which transforms the concatenated list of word embeddings (active in the current context of size $n-1$) to the hidden layer (of size h). This matrix is $H: (n-1)m \times h$
3. The matrix which maps the hidden layer to unnormalized probabilities for each word in the vocabulary. This matrix is $V: V \times h$
4. The matrix connecting the context word embeddings to the output layer. This connection is optional and has dimensions $W: V \times (n-1)m$. Note: while training using SGD, for a single example only $n-1$ words in the context are active out of words V in the vocabulary. I have also omitted the parameter vectors for the bias terms $b: V \times 1$ when computing unnormalized outputs and when computing the hidden layer ($d: h \times 1$).
5. Hence my current understanding is that number of parameters in NNLM is:
$$\dim(C) + \dim(H) + \dim(V) + \dim(W) = (n-1)m \times V + h \times (n-1)m + V \times h + V \times (n-1)m$$

Thank
you!!!



Lecture-#18

Natural Language Processing

A8706

UNIT-IV

Language modeling

language- specific modelling problems



Learning from Experience, Sharing with Passion

- ❖ Language Modeling for Morphologically Rich Languages
- ❖ Selection of Subword Units
- ❖ Modeling with Morphological Categories
- ❖ Languages without Word Segmentation
- ❖ Spoken versus Written Languages

language- specific modelling problems



Learning from Experience, Sharing with Passion

Language Modeling for Morphologically Rich Languages (MRL)

- ❖ A Morphologically Rich Language (MRL) is one which grammatical relations like Subject, Predicate, Object, etc., are indicated by changes to the words instead of relative position or addition of particles.
- ❖ Morphologically rich languages can be either fusional (like Sanskrit, Russian, Ancient Greek, Latin, Classical Arabic, Old English etc.) or agglutinative (like Hungarian, Finnish, Korean, Japanese, Navajo, Old Tupi, Turkish, Swahili etc.).
- ❖ Stemming is probably the least useful NLP method for morphologically rich languages especially if there are a lot of changes in the stem.
- ❖ For many NLP applications, lemmatization is more useful because it will ensure that all different inflectional forms of a word will be mapped to the same lemma.

language- specific modelling problems



Learning from Experience, Sharing with Passion

Selection of Subword Units

- ❖ Sub-word units are usually morphemes, stems and endings, roots, etc. A language model is a description of language.
- ❖ Although grammar has been the prevalent tool in modelling language for a long time, interest has recently shifted towards statistical modelling.
- ❖ Subword Tokenization is a Natural Language Processing technique(NLP) in which a word is split into subwords and these subwords are known as tokens.
- ❖ This technique is used in any NLP task where a model needs to maintain a large vocabulary and complex word structures.
- ❖ For example, we can split “subword” to “sub” and “word”. In other word we use two vector (i.e. “sub” and “word”) to represent “subword”. You may argue that it uses more resource to compute it but the reality is that we can use less footprint by comparing to word representation.

Modeling with Morphological Categories

- ❖ This simplified word form makes it easier for an algorithm to interpret the words in a document. Morphological segmentation is the process of dividing words into morphemes or the base units of a word.
- ❖ These units are things like free morphemes (which can stand alone as words) and prefixes or suffixes.
- ❖ Morphological categories often correspond. in a general way to phonological, semantic, and syntactic categories, but the. correspondence is not close enough to permit a reduction. Thus, the suffixes. that are added to a noun in Latvian, Spanish or Arapesh (the noun's).

language- specific modelling problems



Learning from Experience, Sharing with Passion

Modeling with Morphological Categories

- NUMBER: Singular, Dual, Plural. ...
- GENDER (natural & grammatical): Masculine, Feminine, Neuter (Animate, Vegetable; AND AGREEMENT. ...
- DEFINITENESS: Definite, Indefinite. ...
- POSSESSION: 1st, 2nd, 3rd; Singular & Plural. ...
- NOUN CLASS (Grammatical gender): Declension types I, II, III, etc. ...
- CASE PARADIGM (DECLENSION)

language- specific modelling problems

Languages without Word Segmentation

- ❖ Word segmentation is the problem of splitting a string of written language into its component words.
- ❖ The purpose of this research is to do a comparative study on different techniques of word segmentation and to identify the best technique that would aid in the extraction of keyword from the text.
- ❖ Phoneme segmentation is the ability to break words down into individual sounds. For example, a child may break the word “sand” into its component sounds – /sss/, /aaa/, /nnn/, and /d/.



language- specific modelling problems



Learning from Experience, Sharing with Passion

Spoken versus Written Languages

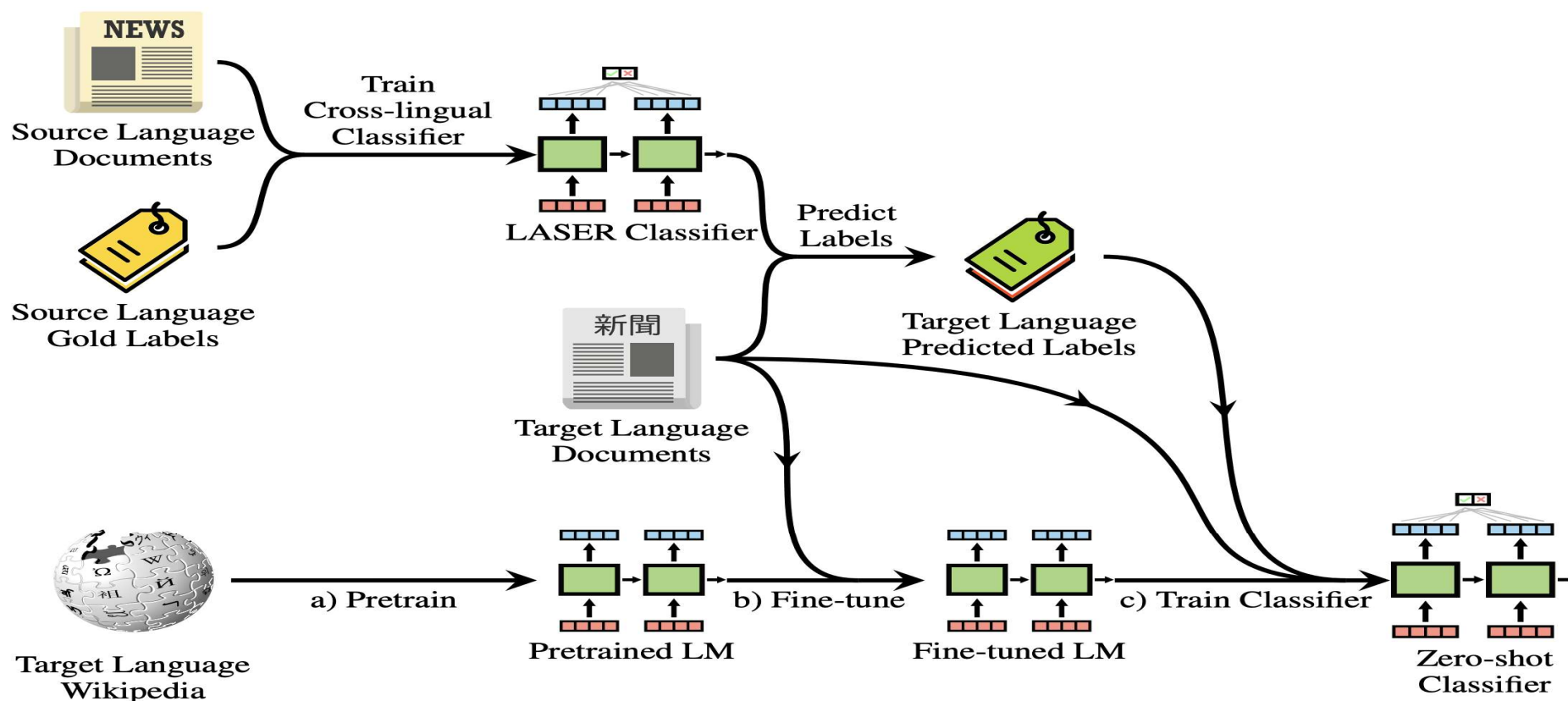
- ❖ **Spoken Language** involves speaking and listening skills.
- ❖ **Written Language** involves reading and writing skills.
- ❖ **Spoken Language** is more informal and simple than written language.
- ❖ **Written Language** is more formal and complex than spoken language.
- ❖ **Spoken Language** is mostly used between two people who are in the same place.
- ❖ **Written Language** promotes communication across space and time.
- ❖ **Spoken Language** contains repetitions, incomplete sentences, interruptions, corrections, etc.
- ❖ **Written Language** is often grammatically correct and may contain long sentences in complex tenses.
- ❖ Example : Son and Sun

Multi-lingual Language Model (MLM)

- ❖ Multilingual Models are a type of Machine Learning model that can understand different languages.
- ❖ One example would be to classify whether a piece of text is a toxic comment.
- ❖ Using a regular Machine learning model we would be able to detect only English language toxic comments but not toxic comments made in Spanish.
- ❖ Multilingual models These models have multilingual analogues—in NLP, models such as mBERT, RemBERT, XLM-RoBERTa, mBART, mT5, and mDeBERTa that were trained in a similar fashion, predicting randomly masked tokens on data of around 100 languages.
- ❖ One such example is Bloom. It is the first multilingual Large Language Model (LLM) trained in complete transparency by the largest collaboration of AI researchers ever involved in a single research project.

Multilingual and Crosslingual Language Modeling

Multi-lingual Language Model (MLM)

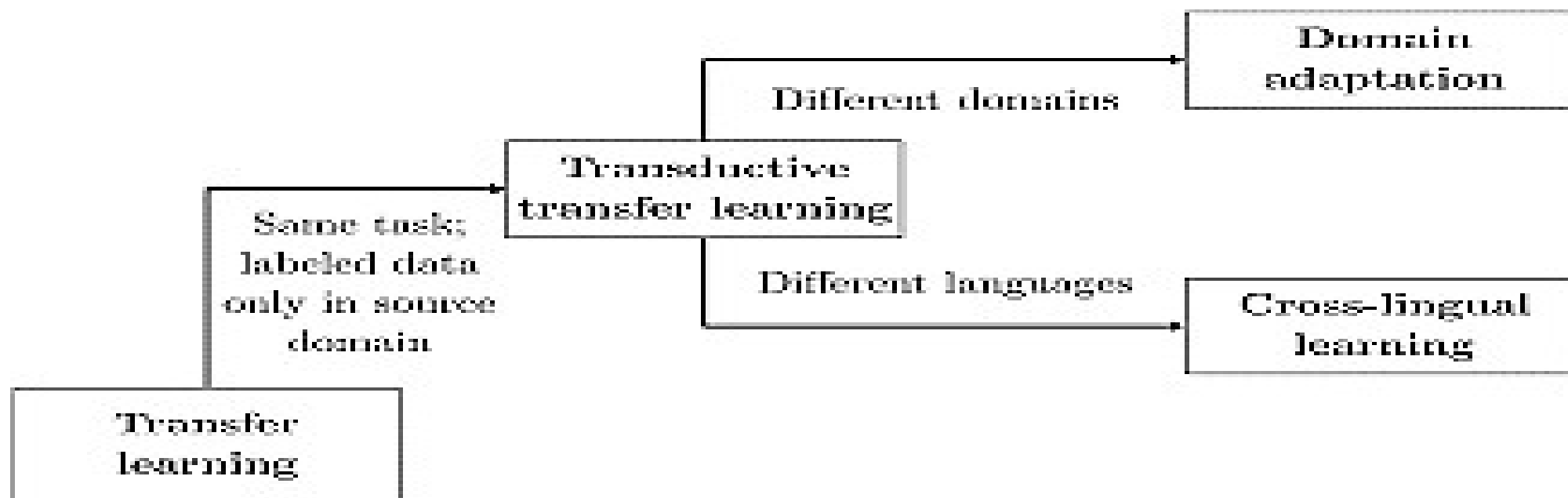


Cross-lingual Language Model (XLM)

- ❖ The model uses the **same shared vocabulary** for all the languages. This helps in establishing a common embedding space for tokens from all languages.
- ❖ Hence, it is evident that languages that have the same script (alphabets), or similar words map better to this common embedding space.
- ❖ For tokenizing the corpora, Byte-Pair Encoding (BPE) is used.
- ❖ Cross-lingual representation learning can be seen as an instance of transfer learning, similar to domain adaptation. The domains in this case are different languages.
- ❖ XLM is a Transformer based architecture that is pre-trained using one of three language modelling objectives: Causal Language Modeling - models the probability of a word given the previous words in a sentence. Masked Language Modeling - the masked language modeling objective of BERT.

Cross-lingual Language Model (XLM)

❖ Cross-lingual representation learning can be seen as an instance of transfer learning, similar to domain adaptation. The domains in this case are different languages.



Cross-lingual Language Model (XLM)

Viewing cross-lingual learning as a form of transfer learning can help us understand why it might be useful and when it might fail:

➤ Transfer learning is useful whenever the scenarios between which we transfer share some underlying structure. In a similar vein, languages share commonalities on many levels—from [loanwords](#) and [cognates](#) on the lexical level, to the [structure of sentences](#) on the syntactic level, to the [meaning of words](#) on the semantic level. Learning about the structure of one language may therefore help us do better when learning a second language.

➤ Transfer learning struggles when the source and target settings are too different. Similarly, it is harder to transfer between languages that are dissimilar (e.g. that have different typological features in the [World Atlas of Language Structures](#)).

Cross-lingual Language Model (XLM)

The XLM model was proposed in [Cross-lingual Language Model Pretraining](#) by Guillaume Lample, Alexis Conneau. It's a transformer pretrained using one of the following objectives:

- a causal language modeling (CLM) objective (next token prediction),
- a masked language modeling (MLM) objective (BERT-like), or
- a Translation Language Modeling (TLM) object (extension of BERT's MLM to multiple language inputs)

Multilingual and Crosslingual Language Modeling



Learning from Experience. Sharing with Passion

	MLM	XLM
1	Multilingual embeddings are happy if the embeddings work well in language A and work well in language B separately without any guarantees about interaction between different languages.	Cross-lingual embeddings attempt to ensure that words that mean the same thing in different languages map to almost the same vector.
2	multi-lingual models are trained on multiple languages simultaneously,	Cross-lingual models are trained on one language and then adapted to other languages.
3	Multi-lingual models are trained on a dataset that includes examples from multiple languages, and they are able to perform tasks such as language identification, machine translation, and text classification in multiple languages.	Cross-lingual models, on the other hand, are trained on a dataset that includes examples from one language, and then they are adapted to other languages using transfer learning techniques.
4	These models can handle multiple languages in one model, but they are not always able to achieve the same high performance as models trained on a single language.	These models can achieve high performance in multiple languages, but they require more data and computational resources to adapt to new languages.

